Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 3.1

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

1. Паттерн Builder. Имеется текст статьи в формате TXT. Статья состоит из заголовка (первая строка), фамилий авторов (вторая строка), самого текста статьи и хеш-кода текста статьи (последняя строка). Написать приложение, позволяющее конвертировать документ в формате TXT в документ формата XML. Необходимо также проверять корректность хеш-кода статьи.

Диаграмма классов

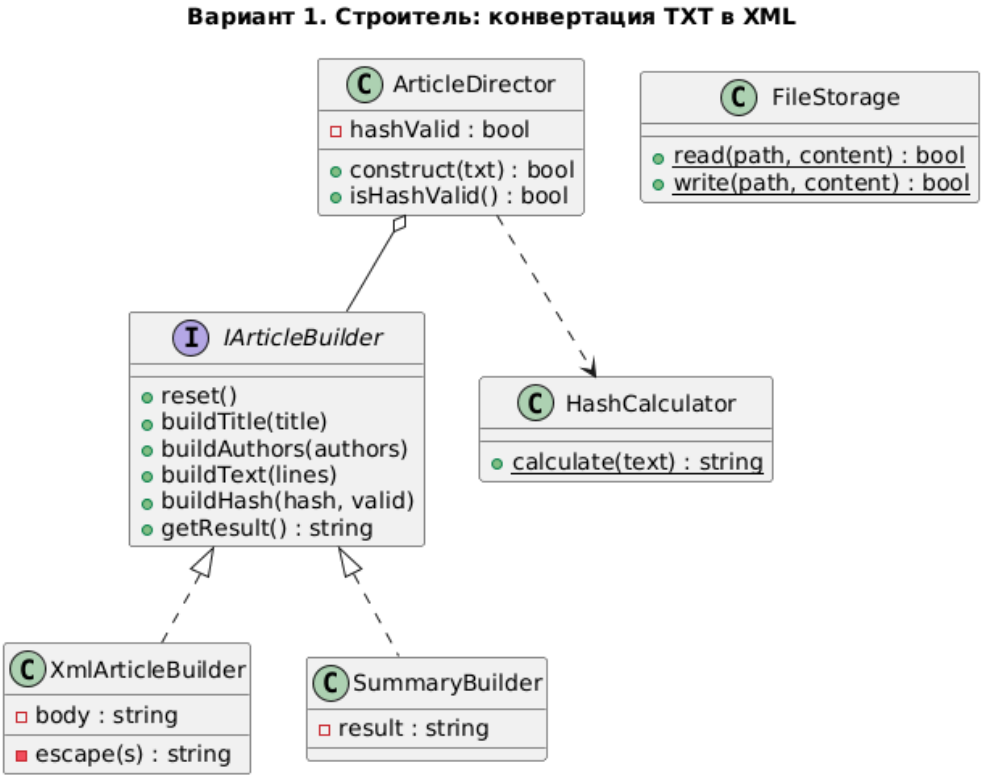
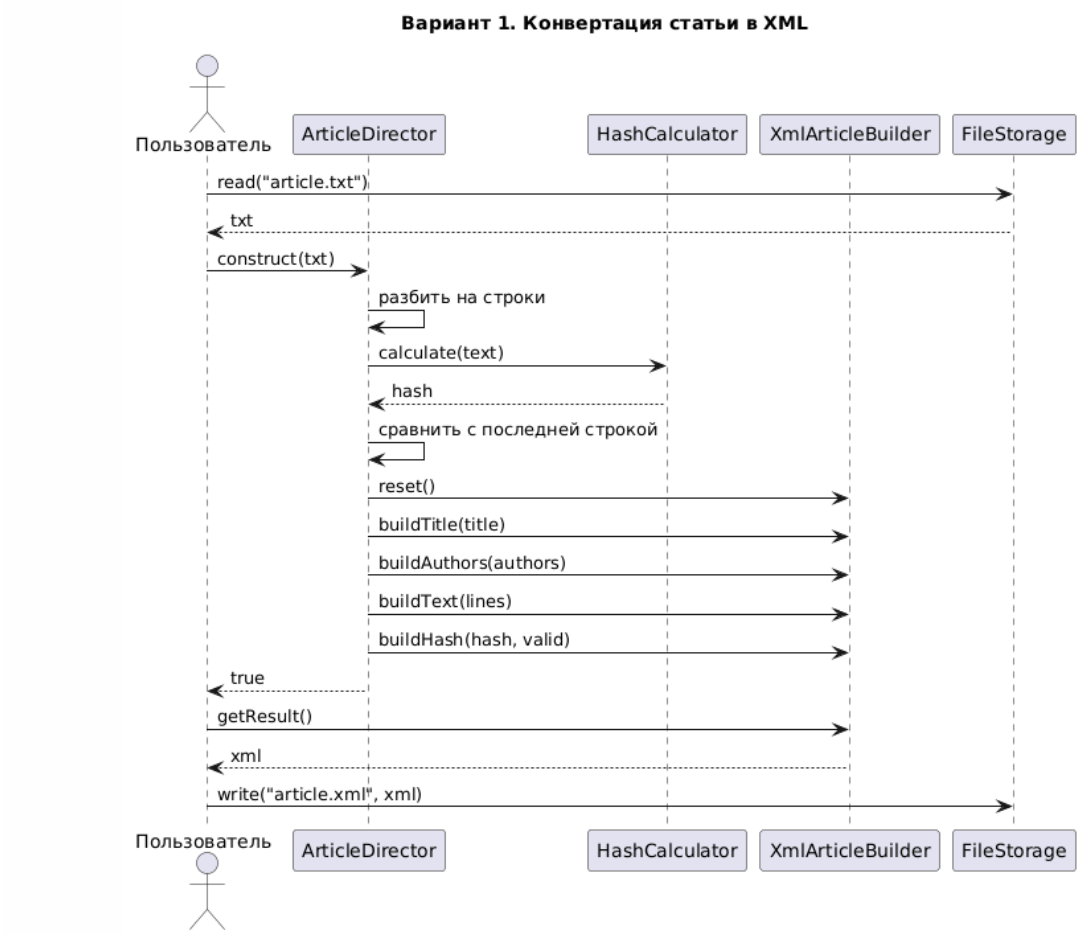


Диаграмма последовательности



Исходный код программы

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <string>
#include <vector>
#include <clocale>

using namespace std;

// ======================== Вспомогательные функции ========================

string trim(const string& s) {
    size_t begin = s.find_first_not_of(" \t\r");
    if (begin == string::npos) return "";
    size_t end = s.find_last_not_of(" \t\r");
    return s.substr(begin, end - begin + 1);
}
```

```
vector<string> splitLines(const string& text) {
    vector<string> lines;
    istringstream in(text);
    string line;
    while (getline(in, line)) {
        if (!line.empty() && line.back() == '\r') line.pop_back();
        lines.push_back(line);
    }
    // Пустые строки в конце файла не считаются
    while (!lines.empty() && trim(lines.back()).empty()) lines.pop_back();
    return lines;
}

vector<string> split(const string& s, char delimiter) {
    vector<string> parts;
    istringstream in(s);
    string part;
    while (getline(in, part, delimiter)) {
        part = trim(part);
        if (!part.empty()) parts.push_back(part);
    }
    return parts;
}

string join(const vector<string>& lines, const string& delimiter) {
    string result;
    for (size_t i = 0; i < lines.size(); i++) {
        if (i > 0) result += delimiter;
        result += lines[i];
    }
    return result;
}

// ======================== Хеш-код ========================

// Хеш FNV-1a (32 бита), записывается 8 шестнадцатеричными цифрами.
class HashCalculator {
public:
    static string calculate(const string& text) {
        unsigned int hash = 2166136261u;
        for (unsigned char c : text) {
            hash ^= c;
            hash *= 16777619u;
        }
        ostringstream out;
        out << hex << setw(8) << setfill('0') << hash;
```

```
        return out.str();
    }
};

// ======================== Строитель (Builder) ========================

// Абстрактный строитель: интерфейс для создания частей документа.
class IArticleBuilder {
public:
    virtual ~IArticleBuilder() {}
    virtual void reset() = 0;
    virtual void buildTitle(const string& title) = 0;
    virtual void buildAuthors(const vector<string>& authors) = 0;
    virtual void buildText(const vector<string>& lines) = 0;
    virtual void buildHash(const string& hash, bool valid) = 0;
    virtual string getResult() const = 0;
};

// Конкретный строитель: собирает документ в формате XML.
class XmlArticleBuilder : public IArticleBuilder {
public:
    void reset() override {
        body = "";
    }

    void buildTitle(const string& title) override {
        body += "  <title>" + escape(title) + "</title>\n";
    }

    void buildAuthors(const vector<string>& authors) override {
        body += "  <authors>\n";
        for (const string& author : authors) {
            body += "    <author>" + escape(author) + "</author>\n";
        }
        body += "  </authors>\n";
    }

    void buildText(const vector<string>& lines) override {
        body += "  <text>\n";
        for (const string& line : lines) {
            if (!trim(line).empty()) {
                body += "    <paragraph>" + escape(trim(line)) + "</paragraph>\n";
            }
        }
        body += "  </text>\n";
    }
```

```
    void buildHash(const string& hash, bool valid) override {
        body += "  <hash valid=\"" + string(valid ? "true" : "false") + "\">" +
                escape(hash) + "</hash>\n";
    }

    string getResult() const override {
        return "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n<article>\n" + body + "</article>\n";
    }

private:
    string body;

    // Замена спецсимволов XML
    static string escape(const string& s) {
        string result;
        for (char c : s) {
            switch (c) {
            case '&': result += "&amp;"; break;
            case '<': result += "&lt;"; break;
            case '>': result += "&gt;"; break;
            case '"': result += "&quot;"; break;
            case '\'': result += "&apos;"; break;
            default: result += c;
            }
        }
        return result;
    }
};

// Второй конкретный строитель: краткий текстовый отчет о статье.
// Показывает, что тот же Распорядитель может строить другое представление.
class SummaryBuilder : public IArticleBuilder {
public:
    void reset() override { result = ""; }

    void buildTitle(const string& title) override {
        result += "Заголовок: " + title + "\n";
    }

    void buildAuthors(const vector<string>& authors) override {
        result += "Авторы (" + to_string(authors.size()) + "): " + join(authors, ", ") + "\n";
    }

    void buildText(const vector<string>& lines) override {
        result += "Строк текста: " + to_string(lines.size()) + "\n";
    }
```

```
    void buildHash(const string& hash, bool valid) override {
        result += "Хеш: " + hash + (valid ? " (верный)" : " (НЕВЕРНЫЙ)") + "\n";
    }

    string getResult() const override { return result; }

private:
    string result;
};

// ======================== Распорядитель (Director) ========================

// Разбирает TXT-статью и по шагам вызывает строителя.
// Не знает, в каком формате строится результат.
class ArticleDirector {
public:
    ArticleDirector(IArticleBuilder& builder) : builder(builder), hashValid(false) {}

    bool construct(const string& txt) {
        vector<string> lines = splitLines(txt);
        if (lines.size() < 4) return false;   // заголовок, авторы, текст, хеш

        string title = trim(lines[0]);
        vector<string> authors = split(lines[1], ',');
        vector<string> text(lines.begin() + 2, lines.end() - 1);
        string hash = trim(lines.back());
        hashValid = HashCalculator::calculate(join(text, "\n")) == hash;

        builder.reset();
        builder.buildTitle(title);
        builder.buildAuthors(authors);
        builder.buildText(text);
        builder.buildHash(hash, hashValid);
        return true;
    }

    bool isHashValid() const { return hashValid; }

private:
    IArticleBuilder& builder;
    bool hashValid;
};

// ======================== Работа с файлами ========================

class FileStorage {
public:
```

```
    static bool read(const string& path, string& content) {
        ifstream in(path, ios::binary);
        if (!in) return false;
        ostringstream buffer;
        buffer << in.rdbuf();
        content = buffer.str();
        return true;
    }

    static bool write(const string& path, const string& content) {
        ofstream out(path, ios::binary);
        if (!out) return false;
        out << content;
        return true;
    }
};

// ======================== Тестирование ========================

class Tester {
public:
    Tester() : passed(0), failed(0) {}

    void check(bool condition, const string& name) {
        if (condition) {
            passed++;
            cout << "  [OK]   " << name << endl;
        } else {
            failed++;
            cout << "  [FAIL] " << name << endl;
        }
    }

    void printResult() const {
        cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
    }

private:
    int passed;
    int failed;
};

bool contains(const string& text, const string& part) {
    return text.find(part) != string::npos;
}

// Собирает TXT-статью с правильным хешем
```

```
string makeArticle(const string& title, const string& authors, const vector<string>& text) {
   return title + "\n" + authors + "\n" + join(text, "\n") + "\n" +
         HashCalculator::calculate(join(text, "\n")) + "\n";
}

void testHash(Tester& t) {
   t.check(HashCalculator::calculate("") == "811c9dc5", "Хеш пустой строки");
   t.check(HashCalculator::calculate("a") == "e40c292c", "Хеш строки \"a\"");
   t.check(HashCalculator::calculate("abc") != HashCalculator::calculate("abd"),
         "Разный текст - разный хеш");
}

void testXmlBuilder(Tester& t) {
   XmlArticleBuilder builder;
   ArticleDirector director(builder);
   string txt = makeArticle("Паттерны", "Иванов, Петров", { "Первая строка.", "Вторая строка." });

   t.check(director.construct(txt), "Статья разобрана");
   t.check(director.isHashValid(), "Правильный хеш принят");
   string xml = builder.getResult();
   t.check(contains(xml, "<title>Паттерны</title>"), "В XML есть заголовок");
   t.check(contains(xml, "<author>Иванов</author>") && contains(xml, "<author>Петров</author>"),
         "В XML есть все авторы");
   t.check(contains(xml, "<paragraph>Вторая строка.</paragraph>"), "В XML есть текст");
   t.check(contains(xml, "valid=\"true\""), "В XML отмечен верный хеш");
}

void testWrongHash(Tester& t) {
   XmlArticleBuilder builder;
   ArticleDirector director(builder);
   string txt = "Заголовок\nИванов\nТекст статьи\n00000000\n";

   t.check(director.construct(txt) && !director.isHashValid(), "Неверный хеш обнаружен");
   t.check(contains(builder.getResult(), "valid=\"false\""), "В XML отмечен неверный хеш");
}

void testBadInput(Tester& t) {
   XmlArticleBuilder builder;
   ArticleDirector director(builder);
   t.check(!director.construct("Заголовок\nИванов\n"), "Слишком короткий файл отклонен");
}

void testEscape(Tester& t) {
   XmlArticleBuilder builder;
   ArticleDirector director(builder);
   director.construct(makeArticle("A & B <C>", "Иванов", { "x < y" }));
   string xml = builder.getResult();
```

```
    t.check(contains(xml, "A &amp;amp; B &amp;lt;C&amp;gt;") && contains(xml, "x &amp;lt; y"),
            "Спецсимволы XML экранируются");
}

void testOtherBuilder(Tester& t) {
    SummaryBuilder builder;
    ArticleDirector director(builder);
    director.construct(makeArticle("Паттерны", "Иванов, Петров", { "Текст." }));
    string summary = builder.getResult();
    t.check(contains(summary, "Авторы (2)") && contains(summary, "(верный)"),
            "Тот же распорядитель строит другое представление");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testHash(t);
    testXmlBuilder(t);
    testWrongHash(t);
    testBadInput(t);
    testEscape(t);
    testOtherBuilder(t);
    t.printResult();
}

// ======================== Демонстрация ========================

// Конвертирует один TXT-текст и печатает результат
void convert(const string& txt, const string& outputPath) {
    XmlArticleBuilder xmlBuilder;
    ArticleDirector xmlDirector(xmlBuilder);
    if (!xmlDirector.construct(txt)) {
        cout << "Ошибка: неверный формат статьи" << endl;
        return;
    }

    SummaryBuilder summaryBuilder;
    ArticleDirector summaryDirector(summaryBuilder);
    summaryDirector.construct(txt);
    cout << summaryBuilder.getResult();

    string xml = xmlBuilder.getResult();
    cout << xml;
    if (FileStorage::write(outputPath, xml)) {
        cout << "Сохранено в файл " << outputPath << endl;
    }
}
```

```
// Запуск: 3_1_Builder.exe article.txt  - конвертирует файл в article.xml.
// Без параметров используется встроенный пример.
int main(int argc, char* argv[]) {
   setlocale(LC_ALL, "Russian");

   cout << "===== Демонстрация =====" << endl;

   if (argc > 1) {
      string txt;
      if (!FileStorage::read(argv[1], txt)) {
         cout << "Не удалось открыть файл " << argv[1] << endl;
         return 1;
      }
      string output = argv[1];
      size_t dot = output.rfind('.');
      if (dot != string::npos) output = output.substr(0, dot);
      convert(txt, output + ".xml");
   } else {
      vector<string> text = {
         "Паттерн Строитель отделяет конструирование сложного объекта",
         "от его представления."
      };
      cout << "--- Статья с правильным хешем ---" << endl;
      convert(makeArticle("Порождающие паттерны", "Иванов, Петров", text), "article.xml");

      cout << "\n--- Статья с испорченным хешем ---" << endl;
      convert("Порождающие паттерны\nИванов, Петров\n" + join(text, "\n") + "\n12345678\n",
            "article_bad.xml");
   }

   runTests();
   return 0;
}
```